

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Sergiu Gherasim, FAF-242

Report

Laboratory Work No. 3

Cryptography and Security

Topic: Polyalphabetic Ciphers

The Playfair Cipher for the Romanian Alphabet

Checked by:
[**Name Surname**], *university assistant*

Chişinău – 2026

CONTENTS

Introduction **3**

1 The Playfair cipher **4**
1.1 The classic algorithm 4
1.2 Adapting it to the Romanian alphabet 4

2 Implementation **6**
2.1 Reading the input 8
2.2 The matrix 9
2.3 Preparing the pairs 9
2.4 Encryption and decryption 9

3 Results **10**
3.1 Encryption 10
3.2 Decryption 11
3.3 Invalid input 12

Conclusions **13**

INTRODUCTION

A monoalphabetic cipher always replaces a letter with the same symbol, so the frequency distribution of the ciphertext is just the distribution of the language, shuffled. Polyalphabetic ciphers break this link: the same plaintext letter is encrypted differently depending on its position or its context, which flattens the distribution and leaves the cryptanalyst much less to work with.

The Playfair cipher, invented by Charles Wheatstone in 1854 and promoted by Lord Playfair, belongs to this family in a particular way. It does not encrypt single letters but pairs of letters (digrams), so the substitution of a letter depends on its neighbour. This makes it one of the first ciphers that works on blocks, an idea that is central to modern block ciphers.

The task of this laboratory work (task 3.1) is to implement Playfair for messages written in Romanian, an alphabet of 31 letters. Only the letters `A`–`Z`, `a`–`z` and the Romanian letters with diacritics are accepted; for any other character the user is shown the allowed range. The key must be at least 7 letters long. The user chooses the operation (encryption or decryption), enters the key and the message or the cryptogram, and receives the cryptogram or the decrypted message. Putting the spaces back into the decrypted text is done manually.

The program is written in Python and is submitted as `playfair.py`.

1 THE PLAYFAIR CIPHER

1.1 The classic algorithm

The classic Playfair cipher uses a 5×5 matrix for the 26 letters of the English alphabet, with `J` merged into `I`. Encryption has three steps:

a) **Preparing the text.** The message is written in capital letters without spaces or punctuation and split into pairs. If a pair would contain the same letter twice, a rare letter (`Q`, `X` or `Z`) is inserted between them. If the final length is odd, one more letter is added at the end.

b) **Building the matrix.** Repeated letters are removed from the key, starting with their second occurrence. The matrix is filled row by row, first with the key and then with the remaining letters of the alphabet in order.

c) **Encrypting the pairs.** Each pair is replaced using the matrix:

- if the two letters are in different rows and columns, each is replaced by the letter in its own row but in the column of the other letter (the opposite corners of a rectangle);
- if they are in the same row, each is replaced by the letter to its right, wrapping from the last column to the first;
- if they are in the same column, each is replaced by the letter below it, wrapping from the last row to the first.

Decryption uses the same matrix and the same rules, except that in the same row each letter is replaced by the one to its *left*, and in the same column by the one *above* it. The rectangle rule is its own inverse, so it does not change.

1.2 Adapting it to the Romanian alphabet

The Romanian alphabet has 31 letters:

A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z

They do not fit in a 5×5 matrix. Merging letters would lose information, since `Ă`, `Â` and `Î` change the meaning of words. Instead I used a 6×6 matrix with 36 cells. The 31 letters fill the first 31 cells, and, as the laboratory guide allows, the 5 free cells are filled with other symbols: the digits `1`–`5`. A side effect is that the cryptogram may contain these digits, so the decryption input accepts them in addition to the letters.

For the padding letter I used `X`, which is rare in Romanian. When the doubled letter is itself `X`, the padding letter is `Q`, so the pair never becomes `XX`.

With the key `CRIPTOGRAFIE`, the repeated letters are removed and the key becomes `CRIPTOGAFE`. This gives the matrix below; it is used for all the examples in this report.

C	R	I	P	T	O
G	A	F	E	Ă	Â
B	D	H	Î	J	K
L	M	N	Q	S	Ș
Ț	U	V	W	X	Y
Z	1	2	3	4	5

Table 1: The 6×6 matrix for the key `CRIPTOGRAFIE`.

2 IMPLEMENTATION

```python
ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ"
FILLERS = "12345"
SIZE = 6
PAD = "X"
ALT_PAD = "Q"


def normalize(text):
    # cedilla variants are produced by some keyboards
    text = text.replace("ş", "ș").replace("Ş", "Ș").replace("ţ", "ț").replace("Ţ", "Ț")
    return text.upper().replace(" ", "")


def find_invalid(text, allowed):
    for char in text:
        if char not in allowed:
            return char
    return None


def get_operation():
    while True:
        operation = input("Choose E (encrypt) or D (decrypt): ").upper()

        if operation in ("E", "D"):
            return operation

        print("Invalid operation. Enter E or D.")


def get_key():
    while True:
        key = normalize(input("Enter the key (at least 7 letters): "))
        invalid = find_invalid(key, ALPHABET)

        if invalid is not None:
            print(f"Invalid character: '{invalid}'.")
            print("Only Romanian letters A-Z, a-z, Ă, Â, Î, Ș, Ț are allowed.")
        elif len(key) < 7:
            print("The key cannot be shorter than 7 letters.")
        else:
            return key


def get_text(operation):
    if operation == "E":
        prompt, allowed = "Enter the message: ", ALPHABET
        hint = "Only Romanian letters A-Z, a-z, Ă, Â, Î, Ș, Ț and spaces are allowed."
    else:
        prompt, allowed = "Enter the cryptogram: ", ALPHABET + FILLERS
        hint = "Only Romanian letters A-Z, a-z, Ă, Â, Î, Ș, Ț, digits 1-5 and spaces are allowed."

    while True:
```

```
        text = normalize(input(prompt))
        invalid = find_invalid(text, allowed)

        if invalid is not None:
            print(f"Invalid character: '{invalid}'.")
            print(hint)
        elif not text:
            print("The text cannot be empty.")
        elif operation == "D" and len(text) % 2 != 0:
            print("The cryptogram must have an even number of characters.")
        else:
            return text


def build_matrix(key):
    chars = ""
    for char in key + ALPHABET + FILLERS:
        if char not in chars:
            chars += char

    matrix = [chars[i:i + SIZE] for i in range(0, SIZE * SIZE, SIZE)]
    pos = {}
    for r, row in enumerate(matrix):
        for c, char in enumerate(row):
            pos[char] = (r, c)

    return matrix, pos


def pad_for(char):
    return ALT_PAD if char == PAD else PAD


def prepare(text):
    pairs = []
    i = 0

    while i < len(text):
        a = text[i]

        if i + 1 == len(text):
            pairs.append(a + pad_for(a))
            i += 1
        elif text[i + 1] == a:
            pairs.append(a + pad_for(a))
            i += 1
        else:
            pairs.append(a + text[i + 1])
            i += 2

    return pairs


def split_pairs(text):
    return [text[i:i + 2] for i in range(0, len(text), 2)]

```

```
110
111 def playfair(pairs, matrix, pos, step):
112     result = ""
113
114     for a, b in pairs:
115         r1, c1 = pos[a]
116         r2, c2 = pos[b]
117
118         if r1 == r2:
119             result += matrix[r1][(c1 + step) % SIZE] + matrix[r2][(c2 + step) % SIZE]
120         elif c1 == c2:
121             result += matrix[(r1 + step) % SIZE][c1] + matrix[(r2 + step) % SIZE][c2]
122         else:
123             result += matrix[r1][c2] + matrix[r2][c1]
124
125     return result
126
127
128 operation = get_operation()
129 key = get_key()
130 text = get_text(operation)
131 matrix, pos = build_matrix(key)
132
133 print("\nMatrix:")
134 for row in matrix:
135     print(" ".join(row))
136 print()
137
138 if operation == "E":
139     pairs = prepare(text)
140     print("Pairs: ", " ".join(pairs))
141     print("Result:", playfair(pairs, matrix, pos, 1))
142 else:
143     pairs = split_pairs(text)
144     print("Pairs: ", " ".join(pairs))
145     print("Result:", playfair(pairs, matrix, pos, -1))
```

Listing 1: playfair.py

2.1 Reading the input

All input goes through `normalize`, which turns the text into capital letters and removes the spaces. It also replaces the cedilla letters ş and ţ with the comma-below letters ș and ț, because some keyboard layouts still produce the old forms, and without this step a correct Romanian word would be rejected.

`find_invalid` returns the first character that is not in the allowed set. Each of the three input functions asks again until the input is valid, and every rejection explains what is allowed:

- `get_operation` accepts only `E` or `D`;
- `get_key` accepts only Romanian letters and at least 7 of them;
- `get_text` accepts only letters for a message. For a cryptogram it also accepts the digits `1`–`5`, because they are part of the matrix, and it requires an even length, because a cryptogram

is always made of whole pairs.

2.2 The matrix

`build_matrix` walks over the key, then the alphabet, then the fillers, and keeps each character only the first time it appears. This removes the repeated letters of the key and appends the remaining letters in alphabetical order in a single loop. The resulting 36 characters are cut into 6 rows. A dictionary `pos` maps each character to its row and column, so the position of a letter is found directly instead of by searching the matrix.

2.3 Preparing the pairs

`prepare` reads the message from left to right. If the next two letters differ, they form a pair. If they are the same letter, only the first one is taken and the padding letter is added, so the second letter starts the next pair. A single letter left at the end is completed in the same way. As in the example of the laboratory guide, a doubled letter that falls across two pairs (such as `AC CE` in `ACCESUL`) is not padded, because the two letters are already in different pairs.

2.4 Encryption and decryption

`playfair` implements both operations with one parameter, `step`. For encryption it is $+1$ (to the right and down), for decryption -1 (to the left and up). The modulo 6 wraps the position around the edge of the matrix; in Python `-1 % 6` is `5`, so stepping left from the first column correctly lands on the last one. The rectangle rule does not use `step` at all, since swapping the columns twice gives back the original pair.

3 RESULTS

3.1 Encryption

The message `Accesul la baza de date este interzis` was encrypted with the key `Criptografie`:

```
Choose E (encrypt) or D (decrypt): E
Enter the key (at least 7 letters): Criptografie
Enter the message: Accesul la baza de date este interzis

Matrix:
C R I P T O
G A F E Ă Â
B D H Î J K
L M N Q S Ș
Ț U V W X Y
Z 1 2 3 4 5

Pairs:  AC CE SU LX LA BA ZA DE DA TE ES TE IN TE RZ IS
Result: GRPGMXSȚMGDG1GÎAMDPĂĂQPĂFVPĂC1TN
```

Listing 2: Encryption run

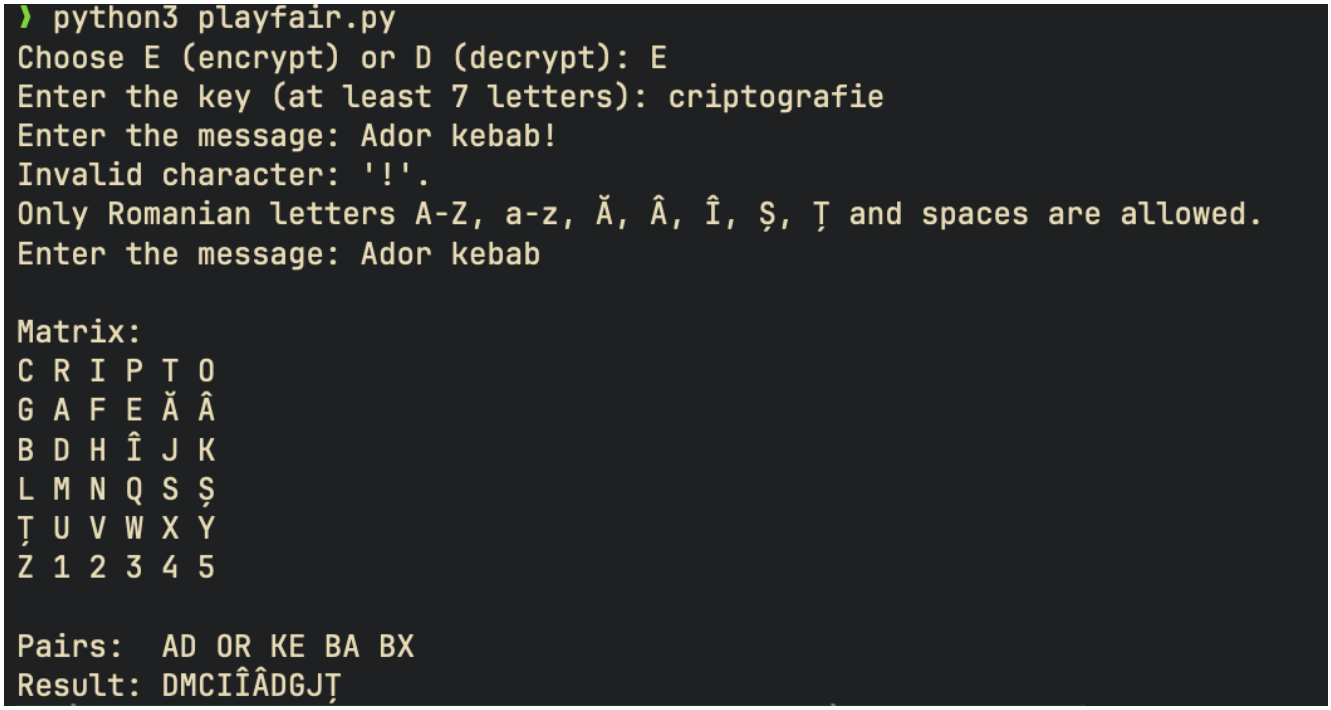


Figure 1: Encrypting the message

The pairs show the preparation step at work. The two `L`s of `ACCESUL LA` would have formed the pair `LL`, so an `X` was inserted and the pair became `LX`. The two `C`s of `ACCESUL` were left alone because they fall into different pairs. A few pairs, checked by hand against the matrix:

- `AC` → `GR`: rectangle rule. `A` is in row 2, column 2 and `C` in row 1, column 1, so `A` becomes the letter of row 2 in column 1 (`G`) and `C` the letter of row 1 in column 2 (`R`).
- `IN` → `FV`: `I` and `N` are both in column 3, so each one is replaced by the letter below it.

- `DA` → `MD`: same column again; `D` moves down to `M` and `A` moves down to `D`.
- `ZA` → `1G`: `Z` is in the last row, so its rectangle corner is a filler cell and the cryptogram contains the digit `1`.

3.2 Decryption

The cryptogram from the previous run was decrypted with the same key:

```
Choose E (encrypt) or D (decrypt): D
Enter the key (at least 7 letters): Criptografie
Enter the cryptogram: GRPGMXSȚMGDG1GÎAMDPĂĂQPĂFVPĂC1TN

Matrix:
C R I P T O
G A F E Ă Â
B D H Î J K
L M N Q S Ș
Ț U V W X Y
Z 1 2 3 4 5

Pairs:  GR PG MX SȚ MG DG 1G ÎA MD PĂ ĂQ PĂ FV PĂ C1 TN
Result: ACCESULXLABAZADEDATEESTEINTERZIS
```

Listing 3: Decryption run

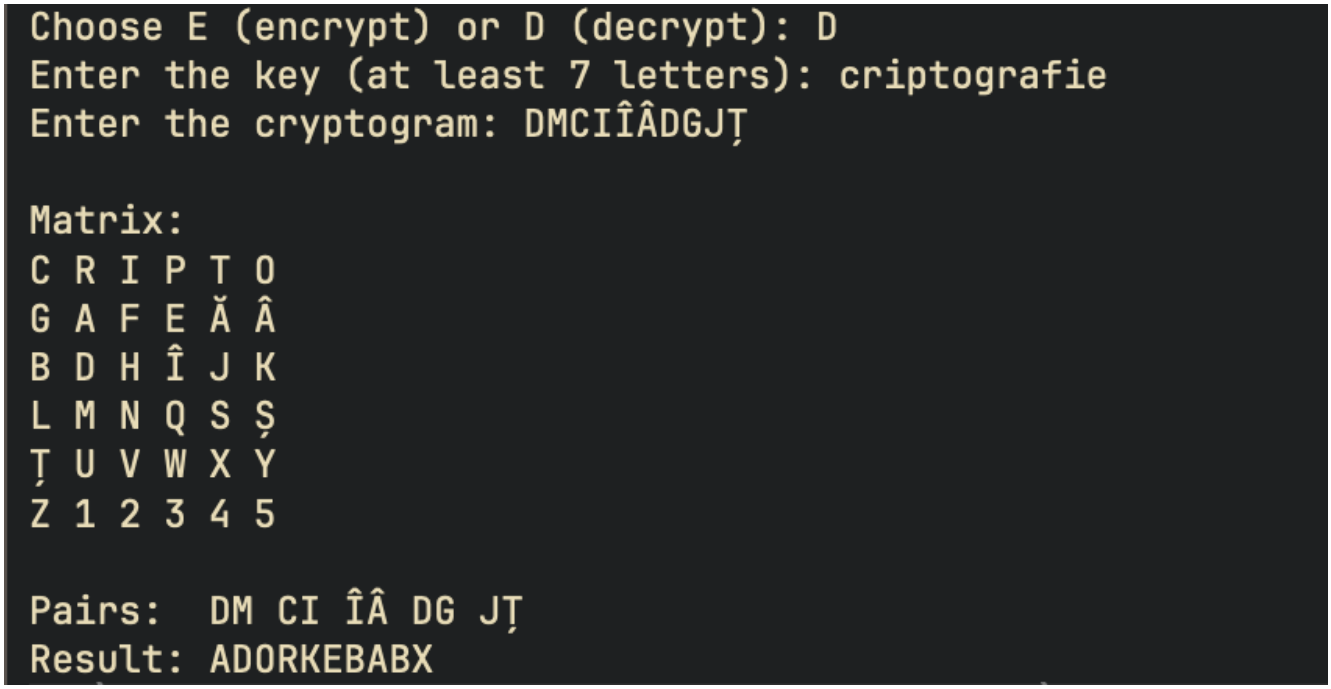


Figure 2: Decrypting the cryptogram

The result is exactly the prepared text, including the inserted `X`. The final step is done manually, as the task requires: the padding letter is removed and the spaces are added back by reading the text, which gives `ACCESUL LA BAZA DE DATE ESTE INTERZIS`.

3.3 Invalid input

```
Choose E (encrypt) or D (decrypt): X
Invalid operation. Enter E or D.
Choose E (encrypt) or D (decrypt): E
Enter the key (at least 7 letters): cheie
The key cannot be shorter than 7 letters.
Enter the key (at least 7 letters): che1e lunga
Invalid character: '1'.
Only Romanian letters A-Z, a-z, Ă, Â, Î, Ș, Ț are allowed.
Enter the key (at least 7 letters): Criptografie
Enter the message: Salut, lume!
Invalid character: ','.
Only Romanian letters A-Z, a-z, Ă, Â, Î, Ș, Ț and spaces are allowed.
Enter the message: Salut lume
...
Pairs:  SA LU TL UM EX
Result: MĂMȚCS1UĂW
```

Listing 4: Rejected inputs

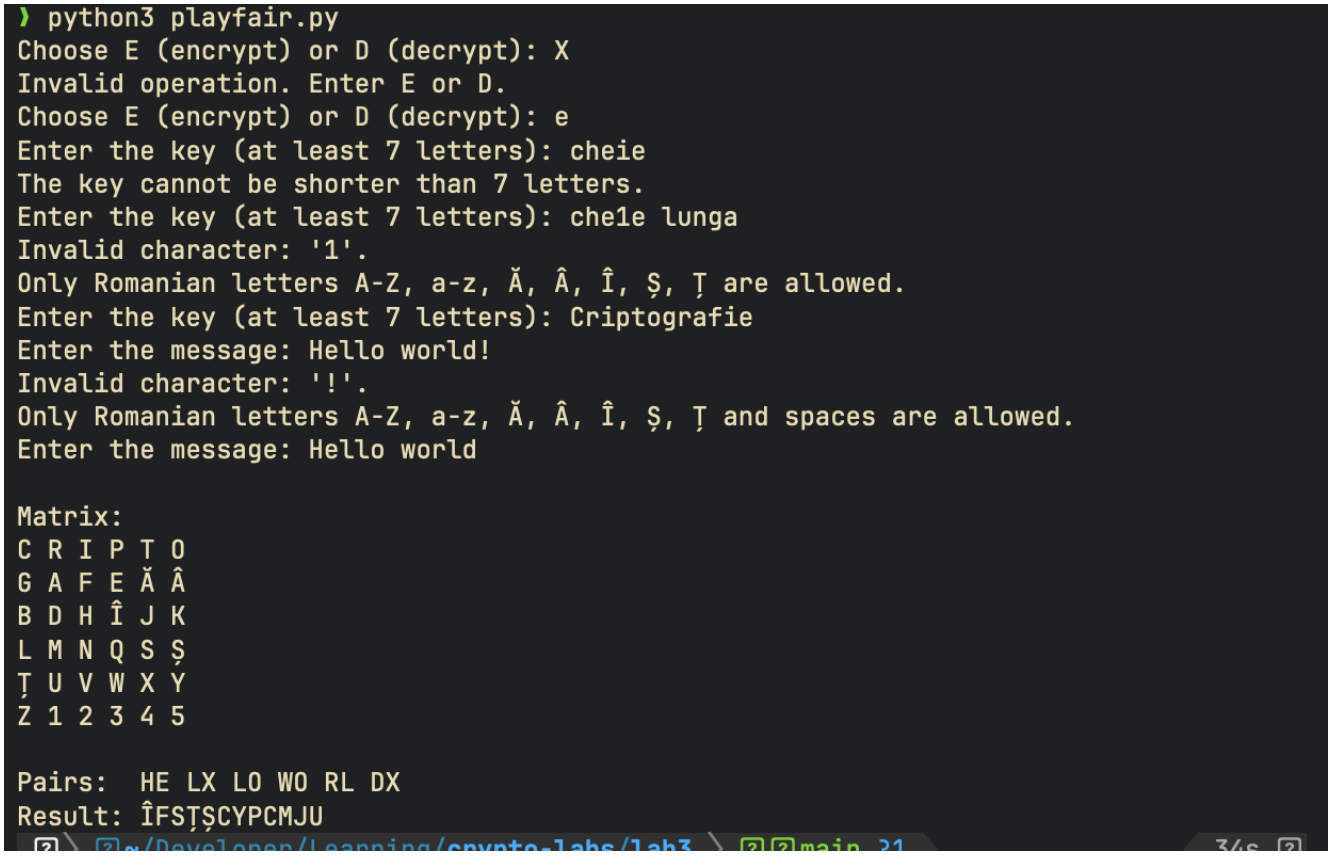


Figure 3: The program rejecting invalid input

Every wrong input is rejected with a message that shows the allowed range of characters, and the program asks again instead of stopping. The last message, `SALUTLUME`, has 9 letters, so an `X` was added to complete the last pair `EX`. In decryption mode an odd-length cryptogram or a character such as `7`, which is not in the matrix, is rejected in the same way.

CONCLUSIONS

In this laboratory work I implemented the Playfair cipher for the Romanian alphabet. The main difficulty was that 31 letters do not fit in the classic 5×5 matrix. Merging letters, as `J` and `I` are merged in English, would have destroyed the diacritics, which carry meaning in Romanian. A 6×6 matrix keeps every letter, and its five free cells are filled with digits. The cost is that the cryptogram can contain digits, so the decryption input has to accept them.

Because Playfair encrypts pairs, the same letter is encrypted differently depending on its neighbour: in the example the letter `A` became `G` in `AC`, `LA` and `BA`, but `D` in `DA`, and in `SA` of the last example it became `Ă`. This hides the single-letter frequencies that make monoalphabetic ciphers easy to break. Digram frequencies, however, are still visible, which is why Playfair is considered insecure today.

The implementation also showed how symmetric the algorithm is. Encryption and decryption are the same function with the direction of the shift reversed, and the rectangle rule is its own inverse. The only step that cannot be undone automatically is the preparation of the text: the padding letters and the spaces have to be restored by the reader, using knowledge of the language.